

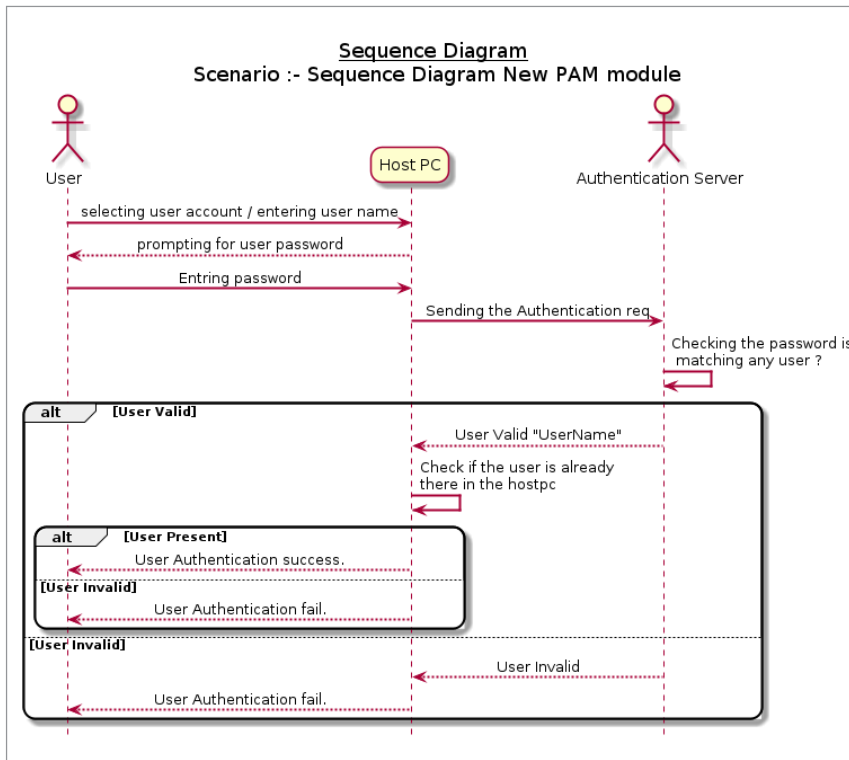


Figure 3: Sequence diagram

user's input. If it does, the user login from this PAM module is a success.

Sequence diagram and code structure

Here are the steps to be followed. First, the user will be prompted to enter the password or key. Once that is done, the PAM module will send the request to the API server, which will validate the details and send the response back to the host PC.

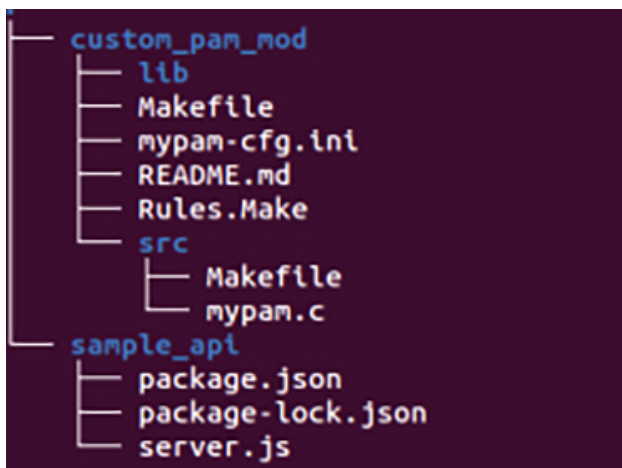


Figure 4: Code structure

The repository shown in Figure 4 contains the source for the server that responds to the API requests from the PAM module, as well as the PAM module source. Once you have completed the build, the output will be stored in the *lib* directory (you can find the repo link at the end of this article).

Setting up an authentication server

This remote server hosts the API, which accepts requests from the host PC and sends authentication responses. Node.js and npm are to be installed in this server.

You can run the *sample_api* with the following command:

```
cd sample_api
node server.js
```

Note: Check the IP address of the server and port that this API service is running. This will be needed to complete the building of the PAM module to work with this server.

Building the PAM module

For building and installing the PAM module, go to the *custom_pam_mod* directory and follow the steps listed below.

First, run the following command to install dependencies for building and running the PAM module.

```
make install_deps
```

Now, modify the build configuration. Inside the INI file (*mypam-cfg.ini*), you'll find various configuration settings specified in key-value pairs. The two main keys you may want to modify to work according to your needs are *server_url* and *replace_logins*.

```
server_url:
```

This key represents the URL of the server. Modify its value to the appropriate server URL.

```
replace_logins:
```

This key controls the replacement of logins. If you want to